Assignment 2

Questions

1. Why does BFS ignore the distances stored in the graph even though they are available to it?
   Answer:
   BFS treats all edges equally and explores nodes level by level without considering their costs. Therefore, it finds the path with the fewest edges (flights), which may not be the path with the shortest total distance.

2. Which two lines change when bfs is turned into dfs, and which of them makes DFS order dependent?
   Answer:
   When BFS is turned into DFS, the first change is state = frontier.popleft() changes to state = frontier.pop(). The second change is for neighbour, _ in graph.get(state,()) changes to for neighbour, _ in reversed(graph.get(state, ())). The line that makes DFS order-dependent is for neighbour, _ in reversed(graph.get(state, ())).

3. UCS tests for the goal when a state is popped from the heap. What could go wrong if it tested when the state was first generated?
   Answer:
   The first path discovered to the goal might be more expensive than another path found later, and testing too early could return that expensive path. With non-negative edge costs, waiting until the goal is popped allows UCS to return the cheapest path.

4. Lab1reports *expanded* and *runtime_ms*. Which one should you use to compare BFS and UCS with a classmate's run on a different laptop, and why?
   Answer:
   I would use expanded because it counts how many states the algorithm explored. This makes it a fairer comparison across different laptops. runtime_ms is affected by processor speed and other programs running on the computer.

5. Which quantity does GBFS ignore that A⋆ uses, and which failure in Lab2 does that explain?
   Answer:
   GBFS ignores g(n), the cost of reaching the current node from the start. It only considers h(n), while A* considers both. This explains why GBFS returned more expensive routes for KUL → KEF and NRT → CPT in Lab 2.

6. Frontier states *X* and *Y* have *(g, h) = (8,2)* and *(3,6)*. Which does UCS expand first? GBFS? A⋆?
   Answer:

- UCS expands Y first because its g value is lower, which is 3 compared with 8.
- GBFS expands X first because its h value is lower, which is 2 compared with 6.
- A* expands Y first because its g + h value is lower, which is 9 compared with 10.

7. Why must *h* be expressed in the same units as the edge costs?
   Answer:
   A* adds g(n) and h(n) to decide which node to explore next. Both values must use the same units for this comparison to make sense. Since the lab's edge costs are in kilometres, h(n) must also be in kilometres.

8. Lab2 computes $h\star$ by running Dijkstra on the reversed graph from the goal. Why reversed?
   Answer:
   This is because reversing the graph allows Dijkstra to start at the goal and calculate the shortest cost from each airport to that goal in the original graph. Without reversing it, Dijkstra would calculate costs travelling out from the goal. These can differ because the graph is directed.

9. Weighted A$\star$ with $w = 1.5$ changed the cost for KUL→KEF but not for SIN→EZE. What property of the SIN→EZE route explains this?
   Answer:
   The optimal SIN → JNB → GRU → EZE route also follows the heuristic's preference for moving closer to the goal. Increasing the heuristic weight to 1.5 still favoured this route, so the cost stayed at 17,817.36 km. This shows that Weighted A* can still find an optimal route, although it is not guaranteed.